

Funcionamiento del Circuito de Respuesta Rápida (Quiz Buzzer)

Generado por Gemini Code Assist

27 de mayo de 2026

1. Introducción

El “Circuito de Respuesta Rápida” o *Quiz Buzzer* es un sistema electrónico diseñado para competencias de conocimiento. Su función principal es el “enclavamiento”: asegurar que el primer participante que presione su botón active su indicador luminoso y bloquee automática e instantáneamente a los demás competidores.

El diseño analizado es robusto y económico, basándose en el uso de tiristores rectificadores controlados de silicio (SCR, como el modelo C106), los cuales actúan como interruptores electrónicos enclavables.

2. Componentes Principales

Para comprender el funcionamiento, es importante destacar el rol de los componentes clave:

- **Tiristores (SCR C106):** Son el corazón del circuito. Permiten el enclavamiento; una vez que reciben un pulso en su compuerta (Gate), entran en conducción y se mantienen encendidos de forma independiente hasta que se les corta la alimentación de voltaje.
- **Red de Diodos (1N4148):** Se utilizan para crear una línea de bloqueo común que tumba el voltaje disponible para las demás compuertas una vez que alguien presiona su botón.
- **Botones Pulsadores Normalmente Abiertos (NA):** Hay uno para cada participante, encargados de enviar el pulso inicial a la compuerta del SCR correspondiente.
- **Botón Pulsador Normalmente Cerrado (NC) / Reset:** Ubicado en la línea principal de alimentación y controlado por el moderador. Sirve para reiniciar el sistema.
- **Indicadores:** Diodos LED para identificar visualmente al estudiante más rápido, y un zumbador (Buzzer) para la señal acústica general.

3. Principio de Funcionamiento Secuencial

La operación del circuito se puede dividir en cinco estados clave:

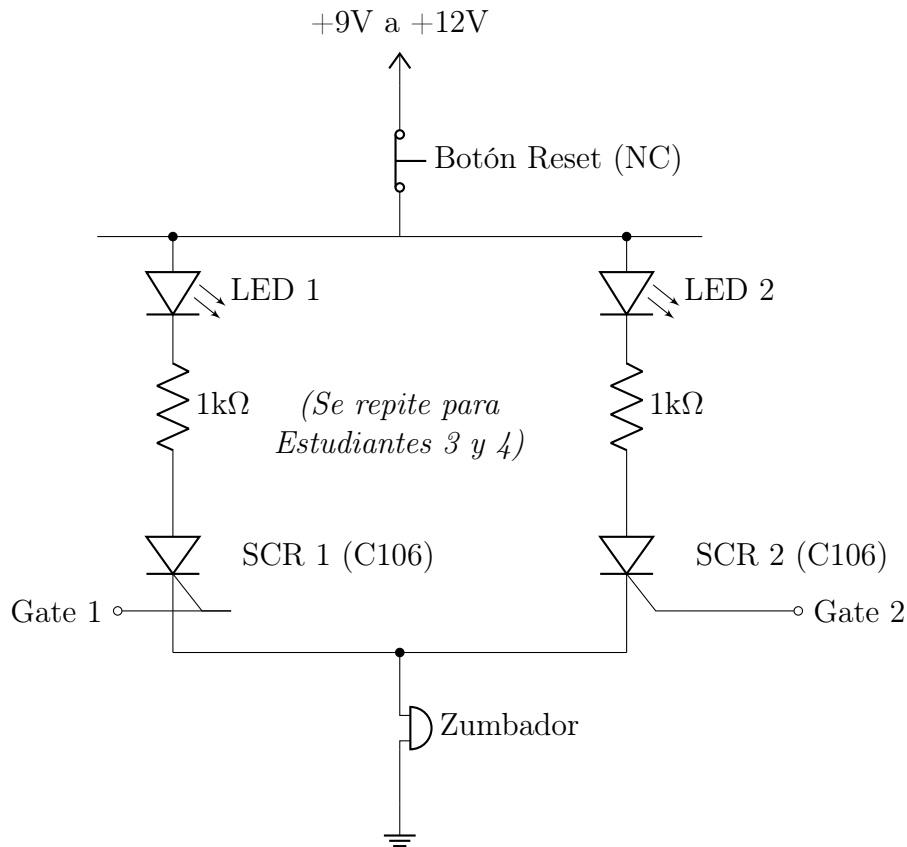
1. **Estado de Espera:** El moderador formula la pregunta. El circuito está energizado (entre 9V y 12V), pero los tiristores están en estado de bloqueo. Ningún LED ni el zumbador están activados porque no hay paso de corriente entre el ánodo y cátodo de los SCRs.
2. **Primer Impacto:** Cuando el primer estudiante (por ejemplo, el participante número 1) reacciona y presiona su botón pulsador, una señal de voltaje viaja instantáneamente a la compuerta (Gate) de su tiristor asignado (SCR 1).
3. **Enclavamiento:** Al recibir este pequeño pulso eléctrico en su compuerta, el SCR 1 se "dispara" y entra en estado de conducción completa. Esto cierra el circuito permitiendo el flujo ininterrumpido de corriente a través de su rama, encendiendo el LED 1 y activando el zumbador común. Debido a las propiedades intrínsecas del tiristor, el componente se mantendrá en conducción indefinidamente, incluso si el estudiante suelta el botón de inmediato.
4. **Bloqueo Absoluto:** Este es el mecanismo crítico que evita empates. Al activarse el SCR 1, este ofrece un camino de muy baja resistencia hacia tierra. Mediante la red común conformada por los diodos (1N4148), el circuito desvía y satura la línea compartida de activación, provocando que el voltaje disponible para los botones restantes caiga a niveles insignificantes. En consecuencia, si los demás estudiantes presionan sus botones microsegundos o milisegundos después, el pulso que envían a sus propios SCRs carece de la energía (tensión) necesaria para activarlos. El circuito ignora así cualquier pulsación tardía de forma puramente analógica.
5. **Liberación (Reset):** Una vez que el participante ha respondido la pregunta y se desea preparar el sistema para la siguiente ronda, el moderador presiona el botón principal de "Reset". Al ser un botón Normalmente Cerrado (NC) en serie con la fuente de poder, pulsarlo corta momentáneamente el suministro de energía a todo el circuito. Al quitar la corriente que fluye entre ánodo y cátodo de los tiristores, estos se "apagan" (salen del estado de enclavamiento). Al soltar el botón, la energía regresa, los LEDs se mantienen apagados, y el sistema está nuevamente en cero, en estado de espera.

4. Esquema del Circuito

A continuación, se presenta la representación gráfica del circuito, generada de forma nativa con `circuitikz`. El diagrama muestra dos de las cuatro ramas para simplificar su visualización, dado que el patrón se repite idénticamente para los estudiantes restantes.

5. Alternativa Moderna: Implementación con Arduino

Para aquellos con conocimientos básicos de programación y microcontroladores, el circuito de respuesta rápida puede ser implementado de manera más compacta y altamente personalizable utilizando una placa de desarrollo como **Arduino Uno** o **Arduino Nano**.



Nota: Las compuertas (Gates) se activan mediante los botones cruzados de los participantes.

Figura 1: Esquema electrónico del Circuito de Respuesta Rápida (basado en SCR C106).

Bajo este enfoque, el funcionamiento lógico (el enclavamiento y el bloqueo cruzado) se traslada por completo del hardware al software, simplificando drásticamente el esquema físico:

- **Entradas (Botones):** Los cuatro botones pulsadores de los participantes se conectan a cuatro pines digitales del Arduino. Por software, estos pines se inicializan usando la directiva `INPUT_PULLUP`. Esto activa las resistencias internas de *pull-up* del microcontrolador, manteniendo el pin en estado alto (`HIGH`) por defecto y permitiendo conectar el otro extremo del botón directamente a tierra (`GND`), sin necesidad de resistencias externas.
- **Salidas (Indicadores):** Los cuatro LEDs (con sus respectivas resistencias limitadoras de corriente) y el zumbador (Buzzer) se conectan a otros pines digitales configurados como salidas (`OUTPUT`).
- **Lógica de Bloqueo (El Código):** El "bloqueo absoluto" que en el circuito analógico hacían los diodos, aquí se logra mediante el uso de una variable booleana de estado (por ejemplo, `bool sistemaBloqueado = false;`). El programa lee continuamente el estado de los botones (esperando un estado `LOW`). En el microsegundo exacto en que detecta la primera pulsación, el código enciende la salida correspondiente, activa el zumbador y cambia la variable a `sistemaBloqueado = true;`.

Mientras esta variable sea verdadera, una condición algorítmica (`if`) se encarga de ignorar cualquier lectura posterior de los demás botones.

- **Reinicio (Reset):** El botón del moderador se puede programar en otro pin de entrada para que, al pulsarlo, devuelva la variable `sistemaBloqueado` a `false` y apague todas las salidas, preparando el sistema para la siguiente ronda; o en su defecto, se puede utilizar el botón de hardware **RESET** incorporado en la propia placa Arduino.

5.1. Análisis del Código (Sketch)

El código implementado en C++ para Arduino (archivo `quiz_buzzer.ino`) replica la lógica de hardware detallada anteriormente. A continuación, se explican sus partes fundamentales:

1. **Definición de pines (`const int`):** Se mapean las conexiones físicas de los componentes a los pines digitales de la placa. Se usan constantes para optimizar el uso de memoria y mejorar la legibilidad.
2. **Variable de estado (`sistemaBloqueado`):** Inicia en `false`. Cumple la misma función que la red de diodos y los tiristores en el esquema analógico: recordar de forma persistente que alguien ya ha presionado un botón.
3. **Bloque `setup()`:** Se utiliza la directiva `INPUT_PULLUP` para activar las resistencias internas del microcontrolador. Al conectar los botones directamente entre el pin y tierra (GND), la lectura cambiará de `HIGH` a `LOW` al ser presionados, evitando el uso de resistencias físicas externas.
4. **Bucle principal (`loop()`):** Un condicional principal `if (!sistemaBloqueado)` garantiza que el microcontrolador solo escuche a los participantes si la ronda está "limpia". En el instante en que detecta una lectura `LOW`, invoca a la función de enclavamiento.
5. **Función `activarGanador()` (Enclavamiento):** Altera la variable `sistemaBloqueado` a `true` y enciende el indicador respectivo. Las lecturas subsiguientes de los demás botones serán ignoradas porque el condicional principal ya no se cumplirá.
6. **Reinicio (Reset):** Esta lógica se ubica fuera de la condición de bloqueo. Independientemente de quién haya ganado, si el moderador presiona su botón, todas las salidas se apagan (`LOW`) y la variable de estado vuelve a `false`, dejando el sistema listo para una nueva ronda.

5.2. Esquema del Circuito con Arduino

A diferencia de la versión analógica, la implementación con Arduino elimina por completo los tiristores y la red de diodos, conectando los componentes directamente a los pines del microcontrolador. Aprovechando las resistencias de *pull-up* internas (`INPUT_PULLUP`), los pulsadores se conectan directamente hacia tierra (GND).

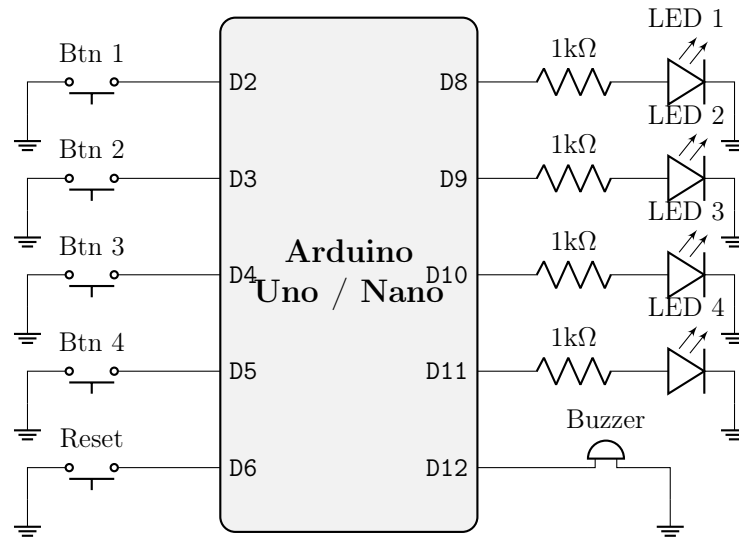


Figura 2: Esquema de conexiones del Circuito de Respuesta Rápida utilizando un Arduino.

6. Evolución Inalámbrica: Implementación con ESP32

Para llevar el *Quiz Buzzer* a un nivel superior, el microcontrolador Arduino puede ser sustituido por un **ESP32**. Este potente SoC (System on a Chip) no solo ejecuta la misma lógica de enclavamiento a gran velocidad, sino que introduce conectividad Wi-Fi y Bluetooth integrada, abriendo la puerta a un sistema IoT completo.

Al migrar el diseño al ESP32, se deben tener en cuenta las siguientes consideraciones y ventajas:

- **Niveles Lógicos de 3.3V:** A diferencia del Arduino Uno que opera a 5V, los pines GPIO del ESP32 funcionan a 3.3V. Es fundamental recalcular el valor de las resistencias limitadoras para los LEDs y asegurarse de que el zumbador (buzzer) sea compatible con este voltaje, o en su defecto, utilizar un transistor para conmutar una fuente de poder externa.
- **Resistencias Pull-up:** Al igual que en Arduino, el ESP32 cuenta con resistencias internas activables por software mediante `INPUT_PULLUP`. Los botones se conectan de la misma manera: un extremo al pin GPIO y el otro a tierra (GND).
- **Pines Táctiles (Touch):** El ESP32 incluye pines con sensores capacitivos integrados. Se podría prescindir por completo de los botones mecánicos y usar simplemente placas metálicas que detecten el toque de los participantes, leyendo las variaciones de capacitancia del cuerpo humano.
- **Conectividad y Marcador Web (Dashboard):** Aprovechando el módulo Wi-Fi y protocolos como WebSockets o MQTT, el ESP32 puede integrarse a una red local. Al ocurrir el "primer impacto", el sistema no solo encendería el LED físico, sino que transmitiría instantáneamente el evento a una pantalla principal mostrando el nombre del estudiante más rápido.

La lógica algorítmica del código base en C++ se mantiene idéntica a la versión de Arduino, facilitando enormemente su migración y escalabilidad.

6.1. Esquema del Circuito con ESP32

El siguiente diagrama ilustra la conexión utilizando el módulo ESP32. Siguiendo el mismo principio de diseño que con Arduino, se aprovechan las resistencias de *pull-up* internas. Nótese que las resistencias limitadoras de corriente para los LEDs se han ajustado a valores menores (por ejemplo, a $220\ \Omega$ o $330\ \Omega$) dado que los pines GPIO proporcionan 3.3V en lugar de los 5V tradicionales.

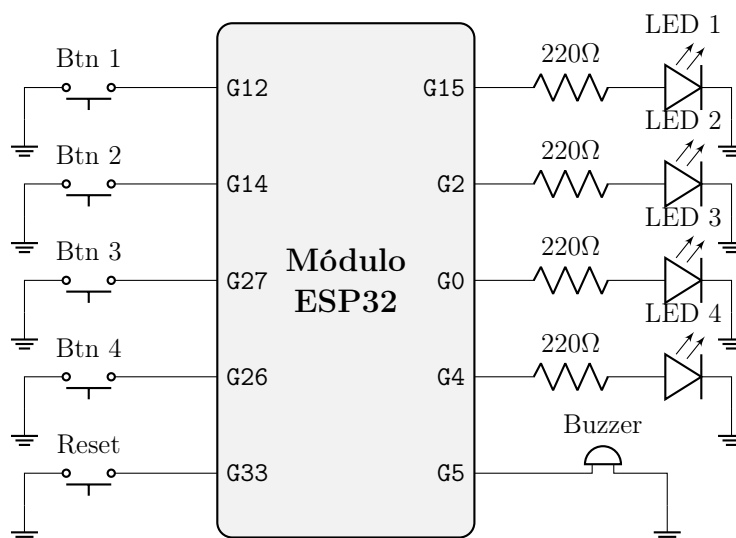


Figura 3: Esquema de conexiones del Circuito de Respuesta Rápida utilizando un ESP32.

6.2. Análisis del Código ESP32 con Servidor Web

El código fuente implementado para el ESP32 (archivo `quiz_buzzer_esp32.ino`) expande la lógica de enclavamiento e integra la conectividad Wi-Fi propia del microcontrolador. Sus partes principales son:

- **Librerías y Conexión Wi-Fi:** Se importan `WiFi.h` y `WebServer.h`. En el bloque `setup()`, el microcontrolador se conecta a una red local especificada mediante el SSID y la contraseña, y expone su dirección IP a través del puerto serie.
- **Pines GPIO:** Se mapean los pines correspondientes a la arquitectura del ESP32 (G12, G14, G27, G26 para entradas y G15, G2, G0, G4 para LEDs).
- **Servidor Web Embebido:** Se define la función `handleRoot()`, que genera y sirve código HTML dinámico. Esta página web incluye una etiqueta `<meta http-equiv="refresh" content="1">` para recargarse automáticamente cada segundo, mostrando el nombre del estudiante almacenado en la variable global `ganador`.
- **Bucle Principal Híbrido:** En el `loop()`, además de verificar instantáneamente qué botón fue presionado (actualizando la variable `ganador`), se ejecuta el comando `server.handleClient()` para mantener vivo el servidor web y despachar la página a cualquier dispositivo (computadora o teléfono) conectado a la red.